RUBIK'S CUBE SOLVER · TEAM UPDATE

# What I tested and what we'll build on

I benchmarked the open-source options for every part of a live, camera-guided Rubik's Cube solver on the same tests. This is the short version; the full numbers are in Benchmark_Report.pdf.

> **Bottom line:** no existing project checks the user's moves live and re-plans after a mistake. We'll combine the best piece for each step and build the move tracking ourselves, and in my tests that tracking worked. **Phase 1** runs on a laptop with its webcam; **Phase 2** then moves it to our XREAL glasses with the Eye camera.

| **21 moves** | **98%** | **100%** | **98.7%** |
|---|---|---|---|
| per solve (today: 110) | of cubes read fully right | cube found by YOLO | moves tracked correctly, 0 wrong |

### How the live solver works



Camera sees cube

→ **YOLO** 7 corner points → **Read 3 sides** 27 sticker colours → **Compare moves** right · wrong · none →

- ✓ next move
- wrong → re-plan
- lost → rescan

kociemba re-plans in ~11 ms after a wrong move

### What we're using

| STEP | CHOICE | RESULT |
|---|---|---|
| Solver | **kociemba** | 20.7 moves vs 110 today · 11 ms |
| Colours | **Centre rebalance + rubiks-color-resolver fallback** | 98% / 88.7% of cubes fully right |
| Finding the cube | **YOLO pose model** (7 corner points) | 100% found · 5.2% point error · 15 ms |
| Reading 3 sides | **7-corner-point method** (idea from sebastienlato/cube-solver) | 27 stickers per frame |
| Move tracking | **Our own**: compare with every possible move | 98.7% / 90.3% correct · 0 wrong moves named |

### What I did

- Reviewed about 20 open-source cube solvers and scanners; ran 8 of them in depth, including the C# AR solver (ported to Python to test it), and re-implemented the 3-side reading method of a 9th.
- Wrote a test-image generator: a 3D cube in real scrambled states with random lighting, glare, fingers, blur and camera noise, so every result is scored automatically.
- Ran the same tests on every option: 200 cubes per solver, 600 frames per detector, 300 whole cubes for colours, 3,600 tracking trials, plus 6 real photos.
- Built and tested the move-tracking logic, and trained a YOLO model (2,500 frames, 32 min) to find the cube for it.
- Checked the XREAL hardware and SDK to see what the glasses can actually run.

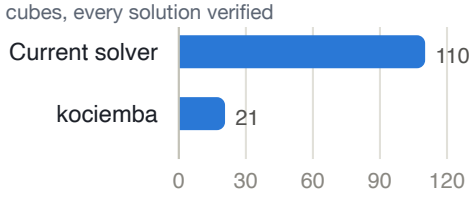
Normal light = even lighting, few fingers in the way. Hard lighting = colour casts, glare, more fingers, blur, noisy camera.

THE EVIDENCE

# Why each choice won

### Solver: moves per solve

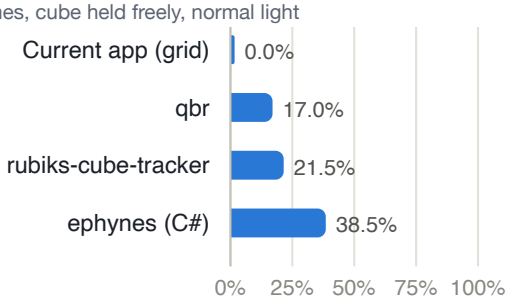
200 random cubes, every solution verified




kociemba solves in about a fifth of the moves, in ~11 ms.

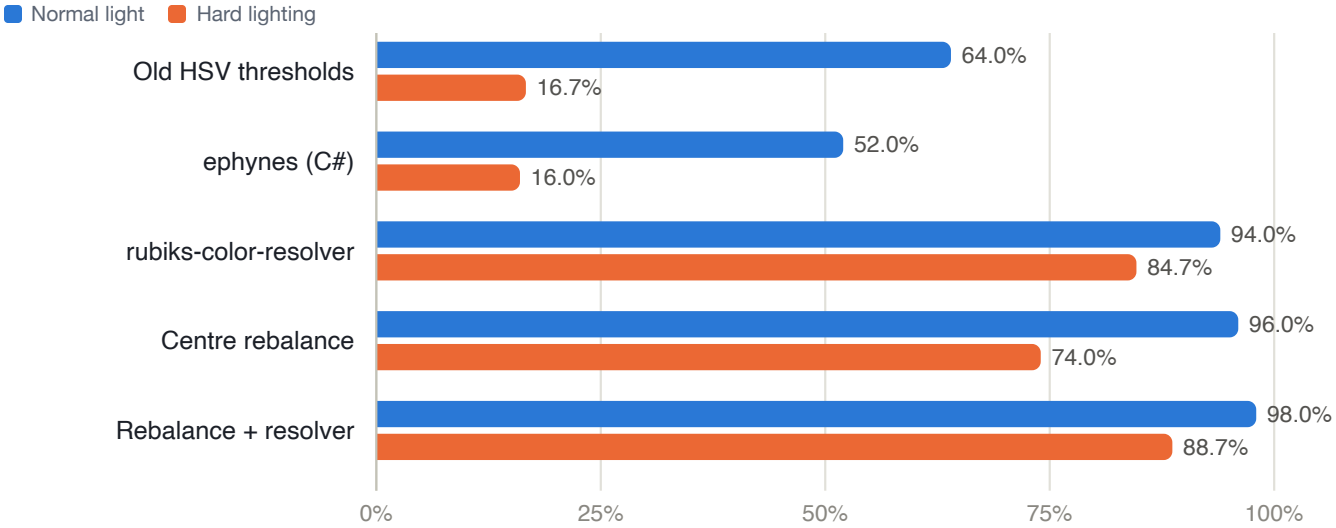
### Finding a hand-held cube face

200 frames, cube held freely, normal light




Classical detectors only work when the cube is lined up with a guide grid. Held freely, the best got 38.5%.

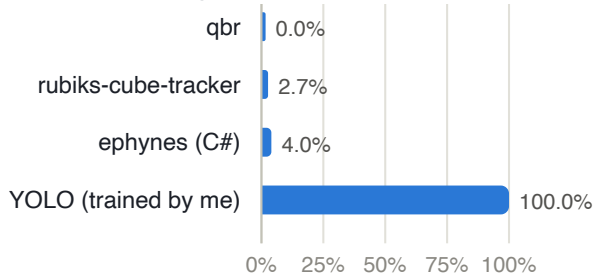
### Naming the colours: % of cubes with all 54 stickers right




Combining our centre rebalance with rubiks-color-resolver beats either alone, especially in hard lighting.

### Finding the cube corner-on (needed for tracking)

150 / 300 frames, normal light



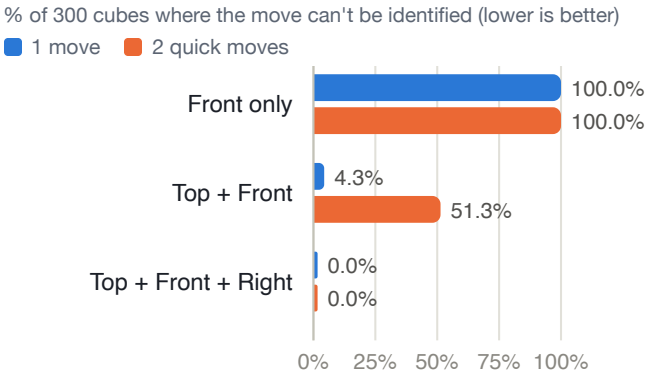

### Why YOLO

Tracking needs the camera to see the cube corner-on. Every classical detector fails at this view (0–4%). The YOLO model I trained finds it in every test frame, in 15 ms on a laptop GPU.

LIVE TRACKING

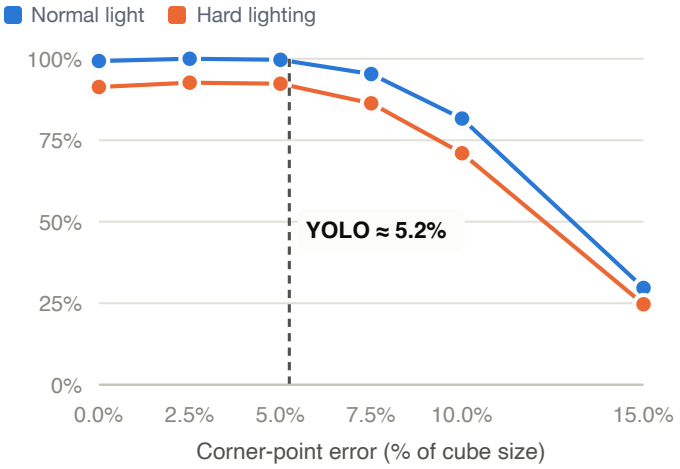
# Checking every move the user makes

### How much of the cube must the camera see?




With only the front visible, turns of the back side are invisible. With **Top + Front + Right**, every move and every pair of quick moves can be identified. So during solving the user holds the cube corner-on to the camera and turns layers only.

### How accurate must the cube's corner points be?

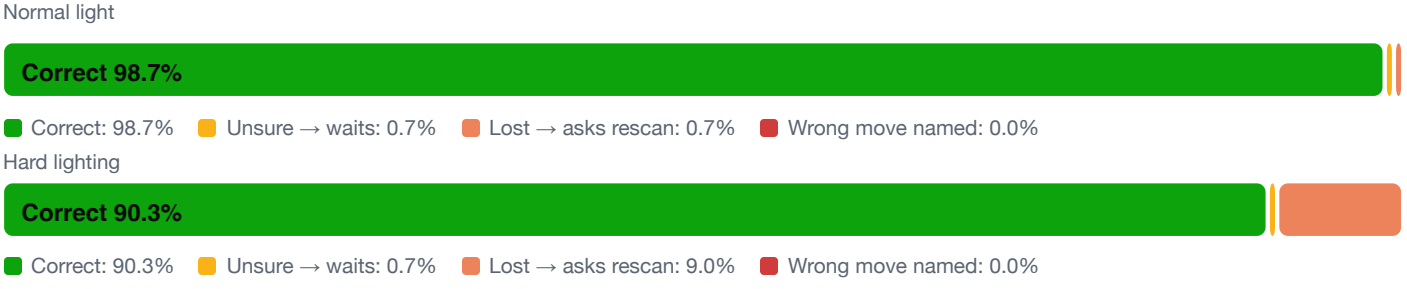



Tracking stays about 99% correct (normal light) while the corner points are within ~5% of the cube's size. YOLO's error is 5.2%, right at that line.

Even when the points are poor it almost never names a wrong move: it says "unsure" and waits, or asks for a rescan.

### Tracking with YOLO finding the cube (300 trials per lighting level)

| | NORMAL LIGHT | HARD LIGHTING |
|---|---|---|
| Correct | 98.7% | 90.3% |
| Wrong move named | 0.0% | 0.0% |
| Unsure / lost (waits or rescans) | 1.3% | 9.7% |



C# AND HARDWARE

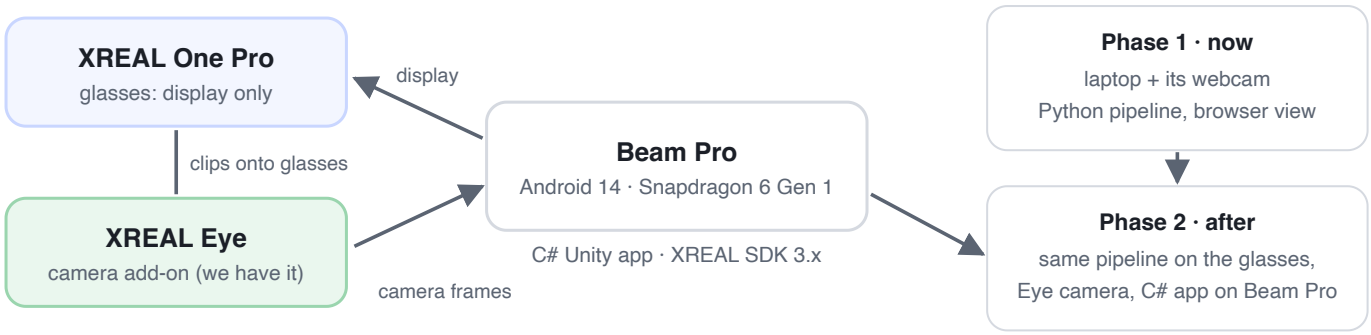
# The C# repo and our glasses

### ephynes/AR-Optimal-Rubiks-Cube-solver (C#): not better

| | EPHYNES (C#) | OUR PIPELINE |
|---|---|---|
| Checks the user's moves | **No** (advances on any stable face) | Yes: 98.7% correct, 0 wrong |
| Finds a hand-held face | 38.5% (grid often rotated) | YOLO: 100% found |
| Colours (whole cube right) | 52% | 98% |
| Solver | 20.7 moves | 20.7 moves |
| Speed per frame | 2.2 ms | YOLO 15 ms (laptop GPU) |

It's fast and has the best classical detector, but it can't check moves, which is our core feature. We may reuse its Unity structure and its C# solver once the solver's licence is confirmed.

### Rollout and hardware




- **Phase 1 (now):** everything runs on a laptop with its webcam. **Phase 2 (once Phase 1 works):** the same pipeline moves to our XREAL One Pro glasses.
- Our glasses have the **XREAL Eye** camera add-on, so in Phase 2 the camera sees exactly what the wearer sees: the ideal view of the cube in their hands.
- Our Unity scaffold targets the old NRSDK; the One Pro needs **XREAL SDK 3.x**.

### Risks and decisions

> **Main risk:** most tests used synthetic images I generated (realistic lighting, glare, fingers, blur). YOLO has only seen synthetic frames, so it needs fine-tuning on a few hundred real photos of our cubes before we trust these numbers.

- **Licences:** kociemba is GPL and YOLO (Ultralytics) is AGPL. Fine for internal use; we need a decision before any release.
- **Phase 2 detail, decided later:** whether the glasses stream frames to the laptop or run everything on the Beam Pro.

### Next steps (Phase 1, on the laptop)

1. Switch to kociemba and the colour fallback.
2. Live camera view with YOLO, then record and label real frames to retrain it.
3. Two-view scan, then live move tracking with re-planning.
4. Then Phase 2: move it to the XREAL glasses with the Eye camera.